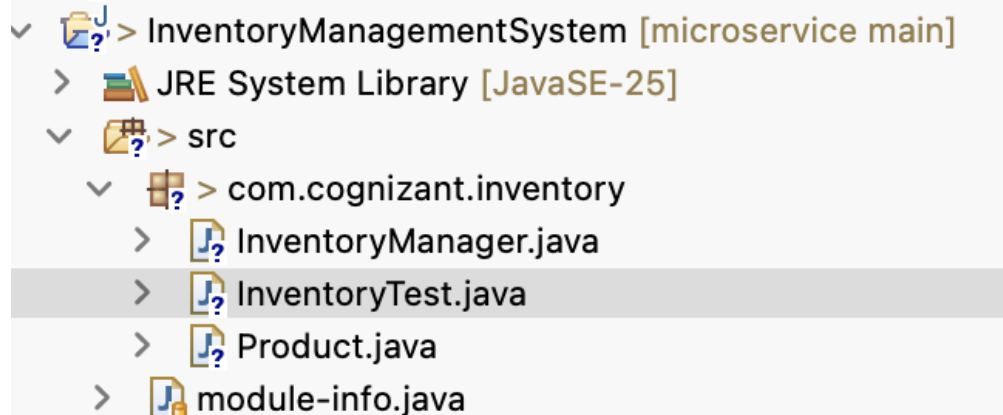


COGNIZANT-DIGITAL-NURTURE-5.0:

Algorithms_DataStructures:

Exercise-1:Inventory Management System



```
package com.cognizant.inventory;

public class InventoryTest {
    public static void main(String[] args) {
        InventoryManager manager=new InventoryManager();

        manager.addProduct(
            new Product(101,
                "Laptop",
                10,
                59000));
        manager.addProduct(
            new Product(102,
                "Mouse",
                50,
                600
            ));
        System.out.println("\nInventory:");

        manager.displayProducts();

        manager.updateProduct(
            101,|
            15,
            58000);
        System.out.println("\nAfter Upadte: ");

        manager.displayProducts();
        manager.deleteProduct(102);

        System.out.println("\nAfter Deletion:");

        manager.displayProducts();
    }
}
```

```

1 package com.cognizant.inventory;
2
3 import java.util.HashMap;
4
5 public class InventoryManager {
6     private HashMap<Integer,Product> inventory=new HashMap<>();
7
8     public void addProduct(Product product) {
9         inventory.put(product.getProductId(), product);
10        System.out.println("product added");
11    }
12
13    public void updateProduct(int productId,int quantity,double price) {
14        Product product = inventory.get(productId);
15        if(product != null) {
16            product.setQuantity(quantity);
17            product.setPrice(price);
18            System.out.println("Product updated");
19        }else {
20            System.out.println("Product Not Found");
21        }
22    }
23
24    public void deleteProduct(int productId) {
25        if (inventory.remove(productId) != null) {
26            System.out.println("Product Deleted");
27        } else {
28            System.out.println("Product Not Found");
29        }
30    }
31
32    public void displayProducts() {
33        for(Product product:inventory.values()) {
34            System.out.println(product);
35        }
36    }
37
38 }
39
40
41 package com.cognizant.inventory;
42
43 public class Product {
44     private int productId;
45     private String productName;
46     private int quantity;
47     private double price;
48
49     public Product(int productId,String productName,int quantity,double price) {
50         this.productId=productId;
51         this.productName=productName;
52         this.quantity=quantity;
53         this.price=price;
54     }
55
56     public int getProductId() {
57         return productId;
58     }
59
60     public void setQuantity(int quantity) {
61         this.quantity=quantity;
62     }
63
64     public void setPrice(double price) {
65         this.price=price;
66     }
67
68     @Override
69     public String toString() {
70         return "Product ID" +productId+
71             ", Name: "+productName +
72             ", Quantity: "+quantity +
73             ", Price: "+price;
74     }
75
76 }
77
78 }
79
80 }

```

```
product added
product added

Inventory:
Product ID101, Name: Laptop, Quantity: 10, Price: 59000.0
Product ID102, Name: Mouse, Quantity: 50, Price: 600.0
Product updated

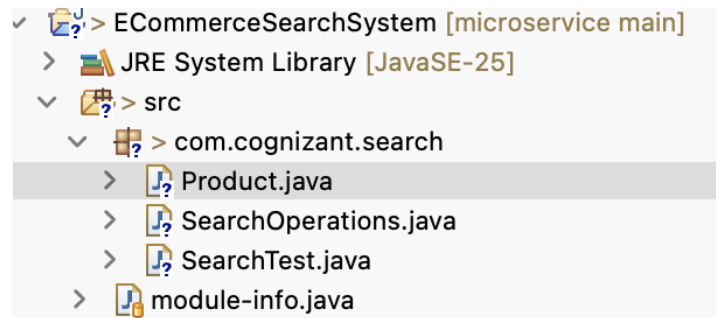
After Upadte:
Product ID101, Name: Laptop, Quantity: 15, Price: 58000.0
Product ID102, Name: Mouse, Quantity: 50, Price: 600.0
Product Deleted

After Deletion:
Product ID101, Name: Laptop, Quantity: 15, Price: 58000.0
```

Output

Exercise2:

E-Commerce Platform Search Function:



```

1 package com.cognizant.search;
2
3 public class Product {
4
5     int productId;
6     String productName;
7     String category;
8
9     public Product(int productId,
10                    String productName,
11                    String category) {
12
13         this.productId = productId;
14         this.productName = productName;
15         this.category = category;
16     }
17
18     @Override
19     public String toString() {
20
21         return "Product ID: " + productId +
22             ", Name: " + productName +
23             ", Category: " + category;
24     }
25 }

```

```

1 package com.cognizant.search;
2
3 public class SearchOperations {
4
5     // Linear Search
6     public static Product linearSearch(
7         Product[] products,
8         String productName) {
9
10        for (Product product : products) {
11
12            if (product.productName.equalsIgnoreCase(
13                productName)) {
14
15                return product;
16            }
17        }
18
19        return null;
20    }
21
22    // Binary Search
23    public static Product binarySearch(
24        Product[] products,
25        String productName) {
26
27        int left = 0;
28        int right = products.length - 1;
29
30        while (left <= right) {
31
32            int mid = (left + right) / 2;
33
34            int comparison =
35                products[mid].productName
36                    .compareToIgnoreCase(
37                        productName);
38
39            if (comparison == 0) {
40
41                return products[mid];
42            } else if (comparison < 0) {
43
44                left = mid + 1;
45            } else {
46
47
48

```

```

1 package com.cognizant.search;
2
3 public class SearchTest {
4
5     public static void main(String[] args) {
6
7         Product[] products = {
8
9             new Product(101,
10                "Camera",
11                "Electronics"),
12
13             new Product(102,
14                "Laptop",
15                "Electronics"),
16
17             new Product(103,
18                "Mobile",
19                "Electronics"),
20
21             new Product(104,
22                "Watch",
23                "Accessories")
24         };
25
26         Product linearResult =
27             SearchOperations.linearSearch(
28                 products,
29                 "Mobile");
30
31         System.out.println(
32             "Linear Search Result:");
33
34         System.out.println(
35             linearResult);
36
37         Product binaryResult =
38             SearchOperations.binarySearch(
39                 products,
40                 "Mobile");
41
42         System.out.println(
43             "\nBinary Search Result:");
44
45         System.out.println(
46             binaryResult);
47     }
48 }

```

output:

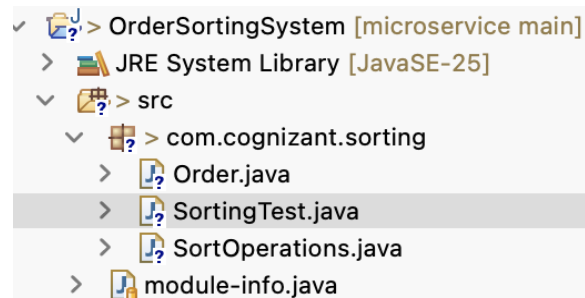
Linear Search Result:

Product ID: 103, Name: Mobile, Category: Electronics

Binary Search Result:

Product ID: 103, Name: Mobile, Category: Electronics

Exercise3:



```

1 package com.cognizant.sorting;
2
3 public class Order {
4
5     int orderId;
6     String customerName;
7     double totalPrice;
8
9     public Order(int orderId,
10                 String customerName,
11                 double totalPrice) {
12
13         this.orderId = orderId;
14         this.customerName = customerName;
15         this.totalPrice = totalPrice;
16     }
17
18     @Override
19     public String toString() {
20
21         return "Order ID: " + orderId +
22             ", Customer: " + customerName +
23             ", Total Price: ₹" + totalPrice;
24     }
25 }

```

Output:

terminated> SortingTest [Java Application] /Library/Java/JavaVirtualMachines

Before Sorting:

```

Order ID: 101, Customer: Sindhu, Total Price: ₹2500.0
Order ID: 102, Customer: Rahul, Total Price: ₹1500.0
Order ID: 103, Customer: Priya, Total Price: ₹4500.0
Order ID: 104, Customer: Arjun, Total Price: ₹3000.0

```

After Quick Sort:

```

Order ID: 102, Customer: Rahul, Total Price: ₹1500.0
Order ID: 101, Customer: Sindhu, Total Price: ₹2500.0
Order ID: 104, Customer: Arjun, Total Price: ₹3000.0
Order ID: 103, Customer: Priya, Total Price: ₹4500.0

```

```
package com.cognizant.sorting;

public class SortingTest {

    public static void main(String[] args) {

        Order[] orders = {

            new Order(
                101,
                "Sindhu",
                2500),

            new Order(
                102,
                "Rahul",
                1500),

            new Order(
                103,
                "Priya",
                4500),

            new Order(
                104,
                "Arjun",
                3000)
        };

        System.out.println(
            "Before Sorting:");

        for (Order order : orders) {

            System.out.println(order);
        }

        SortOperations.quickSort(
            orders,
            0,
            orders.length - 1);

        System.out.println(
            "\nAfter Quick Sort:");

        for (Order order : orders) {

            System.out.println(order);
        }
    }
}
```

```

package com.cognizant.sorting;

public class SortOperations {

    // Bubble Sort
    public static void bubbleSort(Order[] orders) {
        int n = orders.length;

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (orders[j].totalPrice >
                    orders[j + 1].totalPrice) {

                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    // Quick Sort
    public static void quickSort(
        Order[] orders,
        int low,
        int high) {

        if (low < high) {

            int pivotIndex =
                partition(
                    orders,
                    low,
                    high);

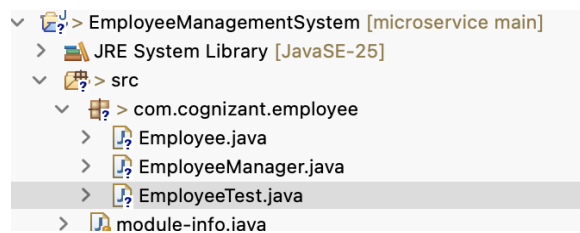
            quickSort(
                orders,
                low,
                pivotIndex - 1);

            quickSort(
                orders,
                pivotIndex + 1,
                high);
        }
    }

    private static int partition(
        Order[] orders,
        int low,

```

Exercise4:



```

package com.cognizant.employee;

public class EmployeeTest {

    public static void main(String[] args) {

        EmployeeManager manager =
            new EmployeeManager(10);

        manager.addEmployee(
            new Employee(
                101,
                "Sindhu",
                "Developer",
                50000));

        manager.addEmployee(
            new Employee(
                102,
                "Rahul",
                "Tester",
                45000));

        manager.addEmployee(
            new Employee(
                103,
                "Priya",
                "Analyst",
                55000));

        System.out.println(
            "\nAll Employees:");

        manager.displayEmployees();

        System.out.println(
            "\nSearching Employee 102:");

        System.out.println(
            manager.searchEmployee(102));

        manager.deleteEmployee(102);

        System.out.println(
            "\nAfter Deletion:");

        manager.displayEmployees();
    }
}

```

```

package com.cognizant.employee;

public class Employee {

    int employeeId;
    String name;
    String position;
    double salary;

    public Employee(int employeeId,
        String name,
        String position,
        double salary) {

        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    @Override
    public String toString() {

        return "Employee ID: " + employeeId +
            ", Name: " + name +
            ", Position: " + position +
            ", Salary: ₹" + salary;
    }
}

```

Output:

```

Employee Added
Employee Added
Employee Added

```

All Employees:

```

Employee ID: 101, Name: Sindhu, Position: Developer, Salary: ₹50000.0
Employee ID: 102, Name: Rahul, Position: Tester, Salary: ₹45000.0
Employee ID: 103, Name: Priya, Position: Analyst, Salary: ₹55000.0

```

Searching Employee 102:

```

Employee ID: 102, Name: Rahul, Position: Tester, Salary: ₹45000.0
Employee Deleted

```

After Deletion:

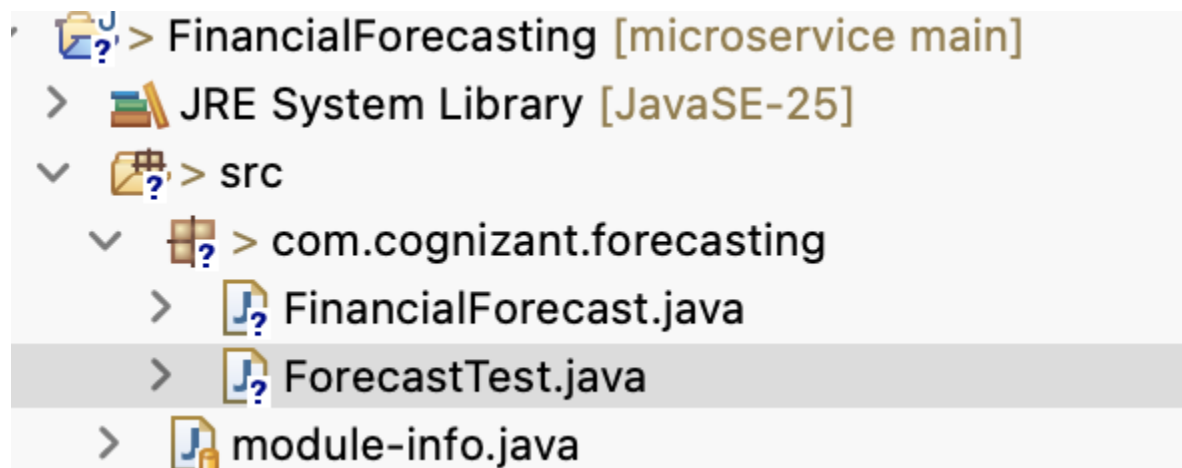
```

Employee ID: 101, Name: Sindhu, Position: Developer, Salary: ₹50000.0
Employee ID: 103, Name: Priya, Position: Analyst, Salary: ₹55000.0

```

Exercise7:

Financial Forecasting (Recursion)



```
package com.cognizant.forecasting;
```

```
public class FinancialForecast {
```

```
    public static double futureValue(  
        double currentValue,  
        double growthRate,  
        int years) {
```

```
        if (years == 0) {  
            return currentValue;  
        }
```

```
        return futureValue(  
            currentValue * (1 + growthRate),  
            growthRate,  
            years - 1);
```

```
    }
```

```
}
```

```

1 package com.cognizant.forecasting;
2
3 public class ForecastTest {
4
5     public static void main(String[] args) {
6
7         double currentValue = 10000;
8
9         double growthRate = 0.10;
10
11         int years = 5;
12
13         double futureValue =
14             FinancialForecast.futureValue(
15                 currentValue,
16                 growthRate,
17                 years);
18
19         System.out.println(
20             "Current Value : ₹" +
21                 currentValue);
22
23         System.out.println(
24             "Growth Rate : " +
25                 (growthRate * 100) + "%");
26
27         System.out.println(
28             "Years : " +
29                 years);
30
31         System.out.println(
32             "Future Value : ₹" +
33                 futureValue);
34     }
35 }

```

output:

```

Current Value : ₹10000.0
Growth Rate   : 10.0%
Years         : 5
Future Value  : ₹16105.100000000008

```